

# ResQSense Wearion

## Database Documentation

Architecture, Schemas, RBAC, Encryption and GDPR

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| <b>Project</b>    | ResQSense Wearion – Firefighter Monitoring System   |
| <b>Version</b>    | 2.0 (with RBAC, pgcrypto, SQLCipher and Audit Log)  |
| <b>Date</b>       | March 2026                                          |
| <b>Databases</b>  | SQLCipher (Raspberry Pi) + PostgreSQL/pgcrypto (PC) |
| <b>Encryption</b> | AES-256 (SQLCipher) + PGP (pgcrypto)                |

# . Table of Contents

1. Introduction and Overview
2. Architecture: SQLCipher vs PostgreSQL + pgcrypto
  - 2.1 Why SQLCipher on the Raspberry Pi
  - 2.2 Why PostgreSQL + pgcrypto on the PC
  - 2.3 Synchronisation Between the Two Databases
3. Complete Table Schema
  - 3.1 role
  - 3.2 team\_leader
  - 3.3 operator
  - 3.4 session / session\_operator
  - 3.5 biometric\_data (with encryption)
  - 3.6 location\_data (with encryption)
  - 3.7 alert
  - 3.8 audit\_log
  - 3.9 failed\_login\_attempt
4. Relationships Between Tables
5. Entity-Relationship (ER) Diagram
6. Class Diagram
7. Roles and Permissions (RBAC)
  - 7.1 Application Roles
  - 7.2 PostgreSQL Users
  - 7.3 Permissions Diagram
8. GDPR Compliance
  - 8.1 GDPR Data Flow
  - 8.2 Encryption: SQLCipher and pgcrypto
  - 8.3 Decryption Functions
9. Indexes and Performance
10. Setup and Usage Guide
  - 10.1 Raspberry Pi (SQLCipher)
  - 10.2 PC (PostgreSQL + pgcrypto)
  - 10.3 Testing Connectivity
  - 10.4 Audit Log Examples

# 1. Introduction and Overview

ResQSense Wearion is a real-time monitoring system for firefighters in emergency operations. The system collects biometric data (heart rate, oxygenation, temperature) and location data (distance, altitude, GPS) through sensor vests worn by operators.

The data architecture uses two distinct databases with encryption at rest and role-based access control (RBAC):

- **SQLCipher on Raspberry Pi 3 Model B** – SQLite with full AES-256 encryption, for real-time operational use in the field.
- **PostgreSQL + pgcrypto on the central PC** – permanent storage with column-level encryption of sensitive data (health and geolocation data).

## What's New in This Version (v2)

|                             |                                                                                    |
|-----------------------------|------------------------------------------------------------------------------------|
| <b>RBAC</b>                 | Roles table with 3 profiles (dev, team_leader, operator) and granular permissions. |
| <b>SQLCipher</b>            | Replacement of SQLite with SQLCipher (AES-256 encryption of the entire DB).        |
| <b>pgcrypto</b>             | Column-level encryption in PostgreSQL for sensitive data (BYTEA).                  |
| <b>Audit Log</b>            | Dedicated table for action traceability (GDPR accountability).                     |
| <b>Decryption functions</b> | biometric_data_decrypted() and location_data_decrypted().                          |
| <b>PostgreSQL Users</b>     | Native DB roles (resqsense_dev, resqsense_leader, resqsense_sync).                 |

## 2. Architecture: SQLCipher vs PostgreSQL + pgcrypto

| Raspberry Pi 3 Model B                                                                                                                                                                                                         |                                                                                                              | PC (Command)                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SQLCipher</b> (AES-256 encrypted SQLite) <ul style="list-style-type: none"> <li>– Real-time data</li> <li>– Local buffer with synced flag</li> <li>– Interface / Dashboard</li> <li>– Full file encryption (.db)</li> </ul> | <ul style="list-style-type: none"> <li>■ Periodic sync (Python via TCP/IP)</li> <li>synced: 0 → 1</li> </ul> | <b>PostgreSQL + pgcrypto</b> <ul style="list-style-type: none"> <li>– Full historical data</li> <li>– Sensitive columns encrypted (BYTEA)</li> <li>– Centralised audit log</li> <li>– Native PostgreSQL roles</li> </ul> |

Figure 1: ResQSense Wearion Data Architecture

### 2.1 Why SQLCipher on the Raspberry Pi

SQLCipher is an SQLite extension that adds transparent AES-256 encryption to the entire database. It was chosen for the Raspberry Pi for the following reasons:

- **Full encryption** – The entire database (including metadata, WAL and journal) is encrypted with AES-256. If the SD card or device is lost or stolen, the data is protected.
- **No server required** – Like SQLite, it requires no server process – it is an embedded library. Ideal for the RPi with only 1 GB of RAM.
- **Field robustness** – Resistant to drops, sudden power loss and vibration. The encrypted .db file handles abrupt failures.
- **Offline operation** – Continues recording data locally even without connectivity, synchronising when possible.
- **Transparency** – The application uses the same SQLite API – just set the key with PRAGMA key before operating.
- **GDPR compliance** – Fulfils Art. 32 of the GDPR (security of processing) with encryption at rest.

#### 'synced' Field –

#### Synchronisation Mechanism

The tables biometric\_data, location\_data, alert, audit\_log and failed\_login\_attempt in SQLite contain a synced (0/1) field that does not exist in PostgreSQL. When synced = 0, the record is pending transmission to the PC.

### 2.2 Why PostgreSQL + pgcrypto on the PC

PostgreSQL with the pgcrypto extension offers column-level encryption, allowing only sensitive data to be encrypted while keeping other fields searchable:

- **Column-level encryption** – Health fields (heart\_rate, oxygenation, temperature) and geolocation (lat, long) are stored as BYTEA encrypted with pgp\_sym\_encrypt.
- **Decryption functions** – SECURITY DEFINER functions (biometric\_data\_decrypted, location\_data\_decrypted) allow querying decrypted data by passing the key as a parameter.
- **Native roles** – PostgreSQL users (resqsense\_dev, resqsense\_leader, resqsense\_sync) with granular table-level permissions (SELECT, INSERT, UPDATE).
- **Scalability** – Supports large volumes of historical data from multiple sessions without degradation.
- **Concurrency** – Multiple users can query data simultaneously without conflicts.
- **Centralised audit log** – The audit\_log table receives records from both local and RPi-synchronised sources.

### 2.3 Synchronisation Between the Two Databases

1. The RPi collects real-time data from the vests and stores it in SQLCipher with synced = 0.
2. Periodically, the sync script queries records with synced = 0.
3. Data is sent to the PC over the internal network (TCP/IP) using the resqsense\_sync user.
4. In PostgreSQL, sensitive data is column-encrypted with pgp\_sym\_encrypt before insertion.
5. After confirmation, the RPi updates the records to synced = 1.

6. A record is created in the PC's audit\_log for traceability.

## Direct Comparison

| Feature          | SQLCipher (Raspberry Pi)        | PostgreSQL + pgcrypto (PC)                                             |
|------------------|---------------------------------|------------------------------------------------------------------------|
| Function         | Operational / Real-time         | Permanent storage                                                      |
| Encryption       | AES-256 total (entire .db file) | Per column (pgp_sym_encrypt)                                           |
| Server           | No (embedded)                   | Yes (service)                                                          |
| DB Roles         | No (application-level control)  | Yes (resqsense_dev, _leader, _sync)                                    |
| synced field     | Yes                             | No                                                                     |
| Encrypted fields | All (total encryption)          | heart_rate_enc, oxygenation_enc,<br>temperature_enc, lat_enc, long_enc |
| Audit log        | Local (synchronised)            | Centralised                                                            |
| Network access   | Local only                      | TCP/IP (internal network)                                              |

## 3. Complete Table Schema

This section describes each table in detail. The schema has been updated with the roles table (RBAC), audit\_log, encrypted columns in PostgreSQL, and SQLCipher support on the Raspberry Pi.

### 3.1 role

Defines the system access profiles (RBAC). Each role specifies granular permissions that control what each type of user can do.

| Field                | Type (PG / SQLite) | Constraints      | Description                            |
|----------------------|--------------------|------------------|----------------------------------------|
| id                   | SERIAL / INTEGER   | PRIMARY KEY      | Profile identifier                     |
| name                 | VARCHAR(50) / TEXT | NOT NULL, UNIQUE | Role name (dev, team_leader, operator) |
| can_view_biometrics  | BOOLEAN / INTEGER  | DEFAULT FALSE/0  | Can view biometric data                |
| can_manage_sessions  | BOOLEAN / INTEGER  | DEFAULT FALSE/0  | Can create/manage sessions             |
| can_manage_operators | BOOLEAN / INTEGER  | DEFAULT FALSE/0  | Can manage operators                   |
| can_manage_roles     | BOOLEAN / INTEGER  | DEFAULT FALSE/0  | Can manage roles (dev only)            |
| can_access_database  | BOOLEAN / INTEGER  | DEFAULT FALSE/0  | Direct DB access (dev only)            |
| description          | TEXT               | Nullable         | Profile description                    |

Pre-defined roles inserted automatically:

| ID | Name        | Biometrics | Sessions | Operators | Roles | Database |
|----|-------------|------------|----------|-----------|-------|----------|
| 1  | dev         | Yes        | Yes      | Yes       | Yes   | Yes      |
| 2  | team_leader | Yes        | Yes      | Yes       | No    | No       |
| 3  | operator    | No         | No       | No        | No    | No       |

### 3.2 team\_leader

Team leaders who authenticate and manage monitoring sessions. Now include role\_id and is\_active.

| Field      | Type (PG / SQLite)  | Constraints             | Description                             |
|------------|---------------------|-------------------------|-----------------------------------------|
| id         | SERIAL / INTEGER    | PRIMARY KEY             | Unique identifier                       |
| name       | VARCHAR(100) / TEXT | NOT NULL                | Full name                               |
| pin_hash   | VARCHAR(255) / TEXT | NOT NULL                | PIN hash (never plain text)             |
| role_id    | INTEGER             | NOT NULL, FK, DEFAULT 2 | Assigned profile (default: team_leader) |
| is_active  | BOOLEAN / INTEGER   | DEFAULT TRUE/1          | Whether the user is active              |
| created_at | TIMESTAMP           | DEFAULT NOW()           | Creation date (PostgreSQL only)         |

Foreign keys: role\_id → role(id)

### 3.3 operator

Firefighters equipped with sensor vests. Now include role\_id.

| Field                | Type (PG / SQLite)  | Constraints                | Description                     |
|----------------------|---------------------|----------------------------|---------------------------------|
| id                   | SERIAL / INTEGER    | PRIMARY KEY                | Unique identifier               |
| name                 | VARCHAR(100) / TEXT | NOT NULL                   | Full name                       |
| vest_id              | VARCHAR(50) / TEXT  | UNIQUE, NOT NULL           | Assigned vest ID                |
| is_alerting          | BOOLEAN / INTEGER   | DEFAULT FALSE/0            | In alert state                  |
| baseline_heart_rate  | INTEGER             | Nullable                   | Baseline heart rate (bpm)       |
| baseline_temperature | FLOAT / REAL        | Nullable                   | Baseline temperature (°C)       |
| baseline_oxygenation | INTEGER             | Nullable                   | Baseline oxygenation (%)        |
| role_id              | INTEGER             | NOT NULL, FK,<br>DEFAULT 3 | Profile (default: operator)     |
| created_at           | TIMESTAMP           | DEFAULT NOW()              | Creation date (PostgreSQL only) |

Foreign keys: role\_id → role(id)

### 3.4 session and session\_operator

Monitoring session and associative table of operators per session. Structure unchanged from v1.

**session:**

| Field      | Type (PG / SQLite)  | Constraints   | Description                  |
|------------|---------------------|---------------|------------------------------|
| session_id | VARCHAR(100) / TEXT | PRIMARY KEY   | Unique identifier (UUID)     |
| leader_id  | INTEGER             | NOT NULL, FK  | Team leader who initiated it |
| started_at | TIMESTAMP / TEXT    | DEFAULT NOW() | Session start                |
| ended_at   | TIMESTAMP / TEXT    | Nullable      | End (NULL = active)          |

**session\_operator:**

| Field       | Type (PG / SQLite)  | Constraints      | Description                  |
|-------------|---------------------|------------------|------------------------------|
| session_id  | VARCHAR(100) / TEXT | PK composite, FK | Session ID                   |
| operator_id | INTEGER             | PK composite, FK | Operator ID                  |
| joined_at   | TIMESTAMP / TEXT    | DEFAULT NOW()    | When they joined the session |

### 3.5 biometric\_data (with encryption)

Biometric data from the sensors. In PostgreSQL, health columns are encrypted with pgcrypto (BYTEA instead of INTEGER/FLOAT). In SQLite, encryption is applied at the entire DB level by SQLCipher.

#### Critical Difference: Column Names

In PostgreSQL, encrypted fields have the \_enc suffix: heart\_rate\_enc, oxygenation\_enc, temperature\_enc (BYTEA type).  
In SQLite, fields keep their original names: heart\_rate, oxygenation, temperature (INTEGER/REAL) because encryption is total.

| Field                         | PostgreSQL Type     | SQLite Type         | Description             |
|-------------------------------|---------------------|---------------------|-------------------------|
| id                            | SERIAL PK           | INTEGER PK AUTO     | Unique identifier       |
| operator_id                   | INTEGER NOT NULL FK | INTEGER NOT NULL FK | Monitored operator      |
| session_id                    | VARCHAR(100) FK     | TEXT FK             | Active session          |
| heart_rate_enc / heart_rate   | BYTEA (encrypted)   | INTEGER             | Heart rate (bpm)        |
| oxygenation_enc / oxygenation | BYTEA (encrypted)   | INTEGER             | O2 saturation (%)       |
| temperature_enc / temperature | BYTEA (encrypted)   | REAL                | Body temperature (°C)   |
| processing_delay_sec          | INTEGER             | INTEGER             | Processing delay (s)    |
| recorded_at                   | TIMESTAMP NOT NULL  | TEXT DEFAULT        | When recorded by sensor |
| received_at                   | TIMESTAMP DEFAULT   | – (n/a)             | Reception at server     |
| synced                        | – (n/a)             | INTEGER DEFAULT 0   | Synchronisation state   |

### 3.6 location\_data (with encryption)

Operator location data. In PostgreSQL, GPS coordinates are encrypted.

| Field           | PostgreSQL Type     | SQLite Type         | Description                       |
|-----------------|---------------------|---------------------|-----------------------------------|
| id              | SERIAL PK           | INTEGER PK AUTO     | Unique identifier                 |
| operator_id     | INTEGER NOT NULL FK | INTEGER NOT NULL FK | Operator                          |
| session_id      | VARCHAR(100) FK     | TEXT FK             | Session                           |
| distance        | FLOAT               | REAL                | Distance from reference point (m) |
| height          | FLOAT               | REAL                | Altitude (m)                      |
| lat_enc / lat   | BYTEA (encrypted)   | REAL                | GPS latitude                      |
| long_enc / long | BYTEA (encrypted)   | REAL                | GPS longitude                     |
| recorded_at     | TIMESTAMP NOT NULL  | TEXT DEFAULT        | When recorded                     |
| received_at     | TIMESTAMP DEFAULT   | – (n/a)             | Reception at server               |
| synced          | – (n/a)             | INTEGER DEFAULT 0   | Synchronisation                   |

### 3.7 alert

Automatically generated alerts: Man Down, tachycardia, fall, abnormal temperature, etc.

| Field | Type (PG / SQLite) | Constraints       | Description |
|-------|--------------------|-------------------|-------------|
| id    | SERIAL / INTEGER   | PK, AUTOINCREMENT | Identifier  |



| Field       | Type (PG / SQLite)  | Constraints             | Description                      |
|-------------|---------------------|-------------------------|----------------------------------|
| operator_id | INTEGER             | NOT NULL, FK            | Operator who triggered the alert |
| session_id  | VARCHAR(100) / TEXT | NOT NULL, FK            | Session in which it occurred     |
| timestamp   | TIMESTAMP / TEXT    | NOT NULL / DEFAULT      | Moment of alert                  |
| description | TEXT                | NOT NULL                | Description (e.g. 'Man Down')    |
| synced      | – / INTEGER         | DEFAULT 0 (SQLite only) | Synchronisation                  |

### 3.8 audit\_log (NEW)

Audit table for traceability (GDPR accountability). Records all critical actions without exposing health data. Each record includes the active role at the time of the action.

| Field        | Type (PG / SQLite)  | Constraints             | Description                                |
|--------------|---------------------|-------------------------|--------------------------------------------|
| id           | SERIAL / INTEGER    | PK, AUTOINCREMENT       | Identifier                                 |
| user_id      | INTEGER             | Nullable                | User who performed the action              |
| role_id      | INTEGER             | FK → role(id)           | Active profile at the time                 |
| action       | VARCHAR(200) / TEXT | NOT NULL                | Action (e.g. LOGIN_SUCCESS, SESSION_START) |
| target_table | VARCHAR(100) / TEXT | Nullable                | Affected table                             |
| target_id    | VARCHAR(100) / TEXT | Nullable                | Affected record ID                         |
| ip_address   | VARCHAR(45)         | Nullable                | Source IP (PostgreSQL only)                |
| timestamp    | TIMESTAMP / TEXT    | DEFAULT NOW()           | Date and time of action                    |
| synced       | – / INTEGER         | DEFAULT 0 (SQLite only) | Synchronisation                            |

#### Examples of recorded actions:

- LOGIN\_SUCCESS, LOGIN\_FAILED – authentication
- SESSION\_START, SESSION\_END – session management
- OPERATOR\_ADDED, OPERATOR\_REMOVED – operator management
- BIOMETRIC\_VIEWED – query of sensitive data
- ALERT\_CREATED – alert generation

### 3.9 failed\_login\_attempt

Failed login attempts for security and auditing purposes.

| Field        | Type (PG / SQLite) | Constraints                | Description                     |
|--------------|--------------------|----------------------------|---------------------------------|
| id           | SERIAL / INTEGER   | PK                         | Identifier                      |
| leader_id    | INTEGER            | Nullable                   | Target leader (if identifiable) |
| attempted_at | TIMESTAMP / TEXT   | NOT NULL / DEFAULT         | Date and time                   |
| ip_address   | VARCHAR(45)        | Nullable (PostgreSQL only) | Source IP                       |

| Field  | Type (PG / SQLite) | Constraints             | Description     |
|--------|--------------------|-------------------------|-----------------|
| synced | – / INTEGER        | DEFAULT 0 (SQLite only) | Synchronisation |

## 4. Relationships Between Tables

| Source Table         | Field       | Target Table        | Relationship                    |
|----------------------|-------------|---------------------|---------------------------------|
| team_leader          | role_id     | role(id)            | N:1 – Many leaders per role     |
| operator             | role_id     | role(id)            | N:1 – Many operators per role   |
| session              | leader_id   | team_leader(id)     | N:1 – Many sessions per leader  |
| session_operator     | session_id  | session(session_id) | N:M – Associative table         |
| session_operator     | operator_id | operator(id)        | N:M – Associative table         |
| biometric_data       | operator_id | operator(id)        | N:1 – Many records per operator |
| biometric_data       | session_id  | session(session_id) | N:1 – Many records per session  |
| location_data        | operator_id | operator(id)        | N:1 – Many records per operator |
| location_data        | session_id  | session(session_id) | N:1 – Many records per session  |
| alert                | operator_id | operator(id)        | N:1 – Many alerts per operator  |
| alert                | session_id  | session(session_id) | N:1 – Many alerts per session   |
| audit_log            | role_id     | role(id)            | N:1 – Contextualises access     |
| failed_login_attempt | leader_id   | team_leader(id)     | N:1 (implicit)                  |

### Summary of Cardinalities

- One Role can be assigned to many Team Leaders and Operators (1:N).
- One Team Leader can start many sessions (1:N).
- Sessions and operators have an M:N relationship via session\_operator.
- Each operator generates many biometric, location and alert records (1:N).
- The Audit Log references roles to contextualise the access level of each action.

## 5. Entity-Relationship (ER) Diagram

The updated ER diagram includes the role (RBAC) and audit\_log tables, as well as encrypted columns in PostgreSQL. Relationships follow the Mermaid schema defined in the project.

| Entity A    | Entity B             | Cardinality | Description                         |
|-------------|----------------------|-------------|-------------------------------------|
| role        | team_leader          | 1:N         | Assigns permissions to the leader   |
| role        | operator             | 1:N         | Assigns permissions to the operator |
| role        | audit_log            | 1:N         | Contextualises access level         |
| team_leader | session              | 1:N         | Starts sessions                     |
| team_leader | failed_login_attempt | 1:N         | Records failed attempts             |
| session     | session_operator     | 1:N         | Contains operators                  |
| operator    | session_operator     | 1:N         | Participates in sessions            |
| operator    | biometric_data       | 1:N         | Generates biometric data            |
| operator    | location_data        | 1:N         | Generates location data             |
| operator    | alert                | 1:N         | Triggers alerts                     |
| session     | biometric_data       | 1:N         | Groups biometric data               |
| session     | location_data        | 1:N         | Groups location data                |
| session     | alert                | 1:N         | Groups alerts                       |

### Main Data Flow

- Team Leader authenticates (pin\_hash) → audit\_log records LOGIN\_SUCCESS.
- Team Leader creates Session → audit\_log records SESSION\_START.
- Operators are added (session\_operator) → audit\_log records OPERATOR\_ADDED.
- Vests send data → biometric\_data + location\_data (encrypted in PG).
- System detects anomalies → alert (Man Down, tachycardia, etc.).
- Team Leader ends session (ended\_at) → audit\_log records SESSION\_END.
- Sync sends data from RPi to PC → audit\_log records receipt.

# 6. Class Diagram

The UML class diagram shows how database entities map to application classes, including controllers and services.

Figure 2: UML Class Diagram of ResQSense Wearion

(Refer to the original document or project repository for the full UML diagram graphic.)

| Class                 | Responsibility                                                                                                     |
|-----------------------|--------------------------------------------------------------------------------------------------------------------|
| AuthenticationManager | Authenticates team leaders (PIN) and firefighters (vest ID). Records failed attempts and actions in the audit_log. |
| DashboardController   | Main controller. Manages the active session, adds/removes operators, displays details.                             |
| Database (Service)    | Data access service: findUserByID, findVestByID, getOperatorProfile.                                               |
| Role (NEW)            | RBAC access profile with granular permissions (can_view_biometrics, can_manage_sessions, etc.).                    |
| TeamLeader            | Team leader with ID, name, pin_hash and role_id.                                                                   |
| Operator              | Firefighter with identification data, vest, baselines and role_id.                                                 |
| Session               | Monitoring session with methods addFirefighterToTeam and createAlert.                                              |
| BiometricData         | Biometric data: heart_rate, oxygenation, temperature (encrypted in PG).                                            |
| Location              | Location data: distance, height, lat, long (GPS encrypted in PG).                                                  |
| Alert                 | Alert with timestamp, description and operator_id.                                                                 |
| AuditLog (NEW)        | Audit record: user_id, role_id, action, target, timestamp.                                                         |

## 7. Roles and Permissions (RBAC)

The system implements role-based access control (RBAC) at two levels: application-level (role table) and database-level (PostgreSQL users). This follows the principle of least privilege required by the GDPR.

### 7.1 Application Roles

| Role                    | Biometrics | Sessions | Operators | Roles | DB  | Description                                                                   |
|-------------------------|------------|----------|-----------|-------|-----|-------------------------------------------------------------------------------|
| dev (Dev team)          | Yes        | Yes      | Yes       | Yes   | Yes | Full access. Can modify DB structure, manage roles and view all data.         |
| team_leader (Team lead) | Yes        | Yes      | Yes       | No    | No  | Manages sessions, adds/removes firefighters, views biometric data and alerts. |
| operator (Firefighter)  | No         | No       | No        | No    | No  | Can only view their own basic data. No access to health data or management.   |

### 7.2 PostgreSQL Users

| PG User          | Corresponding Role | Permissions                                                                       | Purpose                                                        |
|------------------|--------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------|
| resqsense_dev    | dev                | ALL PRIVILEGES                                                                    | Full access to all tables and sequences. Can modify structure. |
| resqsense_leader | team_leader        | SELECT, INSERT, UPDATE on operational tables; SELECT on data; INSERT on audit_log | Day-to-day operations. Cannot delete data.                     |
| resqsense_sync   | – (sync)           | INSERT on data and audit_log; SELECT on reference tables                          | Used by RPi to send data. Insert only, no modification.        |

#### Dual Security Layer

Access control operates at two layers: the application checks the user's role before executing actions (role table), and PostgreSQL enforces additional restrictions at the database level (GRANT/REVOKE). Even if the application has a vulnerability, PostgreSQL prevents unauthorised operations.

### 7.3 Tables Accessible by Role

| Table       | dev                    | team_leader            | operator     |
|-------------|------------------------|------------------------|--------------|
| role        | ALL                    | SELECT                 | –            |
| team_leader | SELECT, INSERT, UPDATE | SELECT, INSERT, UPDATE | –            |
| operator    | SELECT, INSERT, UPDATE | SELECT, INSERT, UPDATE | SELECT (own) |
| session     | ALL                    | SELECT, INSERT, UPDATE | –            |

| Table                | dev | team_leader            | operator |
|----------------------|-----|------------------------|----------|
| session_operator     | ALL | SELECT, INSERT, UPDATE | –        |
| biometric_data       | ALL | SELECT                 | –        |
| location_data        | ALL | SELECT                 | –        |
| alert                | ALL | SELECT, INSERT, UPDATE | –        |
| audit_log            | ALL | INSERT                 | –        |
| failed_login_attempt | ALL | INSERT                 | –        |

## 8. GDPR Compliance

The processing of personal and sensitive data in ResQSense Wearion follows the principles of the General Data Protection Regulation (GDPR).

### 8.1 GDPR Data Flow

|                       |                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1. Vest / Sensor      | Generates BiometricData and Location data transmitted to the Raspberry Pi.                                                 |
| 2. Raspberry Pi       | Receives the data and checks whether an active session exists.                                                             |
| 3. Active session?    | If no active session, data is immediately discarded (ignored).                                                             |
| 4. GDPR Check         | Legal basis: Vital Interest (Art. 6(1)(d) and Art. 9(2)(c)) – protection of firefighters' lives.                           |
| 5. Minimisation       | Filter only the variables defined in the BiometricData and Location classes. No unnecessary data collected (Art. 5(1)(c)). |
| 6. Pseudonymisation   | Operator.id is separated from health and geolocation data, reducing re-identification risk (Art. 4(5)).                    |
| 7. Encryption at Rest | SQLCipher encrypts the entire database with AES-256 (Art. 32).                                                             |
| 8. Local Storage      | Data stored in SQLite with synced=0 flag. Interface displays real-time data.                                               |
| 9. Local Audit Log    | Action recorded without exposing health metrics.                                                                           |
| 10. Synchronisation   | When network is available, the sync script sends records with synced=0 to the PC.                                          |
| 11. pgcrypto (PC)     | Sensitive columns are individually encrypted with pgp_sym_encrypt before insertion into PostgreSQL.                        |
| 12. Permanent Storage | PostgreSQL stores the full history with column-encrypted data.                                                             |
| 13. Central Audit Log | Data reception recorded in the centralised audit_log.                                                                      |
| 14. synced=1 Marking  | On the RPi, synchronised records are marked as synced=1.                                                                   |

### 8.2 Encryption: SQLCipher and pgcrypto

| Aspect           | SQLCipher (Raspberry Pi)    | pgcrypto (PostgreSQL)                                               |
|------------------|-----------------------------|---------------------------------------------------------------------|
| Encryption type  | Total (entire .db file)     | Per column (BYTEA fields)                                           |
| Algorithm        | AES-256-CBC                 | PGP symmetric (AES)                                                 |
| Encrypted fields | All (transparent)           | heart_rate_enc, oxygenation_enc, temperature_enc, lat_enc, long_enc |
| Key              | PRAGMA key on open          | Parameter in pgp_sym_encrypt/decrypt functions                      |
| Key management   | Environment variable / .env | Environment variable / .env                                         |
| Performance      | Minimal (transparent)       | Overhead on encryption/decryption per query                         |
| Searchability    | Fields normally searchable  | Encrypted fields not directly searchable                            |



### 8.3 Decryption Functions (PostgreSQL)

The PostgreSQL schema includes SECURITY DEFINER functions to decrypt data in a controlled manner:

**biometric\_data\_decrypted(encryption\_key TEXT)**

Returns all biometric data with decrypted health fields. Receives the encryption key as a parameter.

**location\_data\_decrypted(encryption\_key TEXT)**

Returns all location data with lat and long decrypted.

**Usage example:**

```
-- Query decrypted biometric data
SELECT * FROM biometric_data_decrypted('SECRET_KEY');

-- Insert encrypted data
INSERT INTO biometric_data (operator_id, session_id,
heart_rate_enc, oxygenation_enc, temperature_enc,
processing_delay_sec, recorded_at)
VALUES (1, 'session_001',
pgp_sym_encrypt('90', 'SECRET_KEY'),
pgp_sym_encrypt('97', 'SECRET_KEY'),
pgp_sym_encrypt('37.2', 'SECRET_KEY'),
1, '2025-03-18 16:35:00');
```

### GDPR Principles Applied

| GDPR Principle                      | Implementation in the System                           |
|-------------------------------------|--------------------------------------------------------|
| Legal basis (Art. 6)                | Vital interest – protection of firefighters' lives     |
| Minimisation (Art. 5(1)(c))         | Only strictly necessary variables in the classes       |
| Pseudonymisation (Art. 4(5))        | operator_id separated from health and geolocation data |
| Encryption (Art. 32)                | SQLCipher (AES-256 total) + pgcrypto (per column)      |
| Access control (Art. 32)            | RBAC with 3 roles + native PostgreSQL users            |
| Accountability (Art. 5(2))          | audit_log with user_id, role_id, action and timestamp  |
| Retention limitation (Art. 5(1)(e)) | Data on RPi synchronised and periodically purged       |

# 9. Indexes and Performance

## PostgreSQL Indexes

| Index                       | Table          | Columns                  | Purpose                         |
|-----------------------------|----------------|--------------------------|---------------------------------|
| idx_biometric_operator_time | biometric_data | operator_id, recorded_at | Biometric data by operator/time |
| idx_biometric_session       | biometric_data | session_id               | Filter by session               |
| idx_location_operator_time  | location_data  | operator_id, recorded_at | Location by operator/time       |
| idx_location_session        | location_data  | session_id               | Filter by session               |
| idx_alert_session           | alert          | session_id, timestamp    | Alerts by session               |
| idx_alert_operator          | alert          | operator_id              | Alerts by operator              |
| idx_session_leader          | session        | leader_id                | Sessions by leader              |
| idx_audit_user              | audit_log      | user_id, timestamp       | Actions by user                 |
| idx_audit_action            | audit_log      | action, timestamp        | Actions by type                 |
| idx_team_leader_role        | team_leader    | role_id                  | Leaders by role                 |
| idx_operator_role           | operator       | role_id                  | Operators by role               |

## Additional SQLite Indexes (Synchronisation)

| Index                | Table          | Column | Purpose                 |
|----------------------|----------------|--------|-------------------------|
| idx_biometric_synced | biometric_data | synced | Records pending sync    |
| idx_location_synced  | location_data  | synced | Locations pending sync  |
| idx_alert_synced     | alert          | synced | Alerts pending sync     |
| idx_audit_synced     | audit_log      | synced | Audit logs pending sync |

## 10. Setup and Usage Guide

### 10.1 Raspberry Pi (SQLCipher)

Install SQLCipher:

```
sudo apt update && sudo apt upgrade -y
sudo apt install sqlcipher libsqlcipher-dev -y
sqlcipher --version
```

Install Python dependency:

```
pip3 install pysqlcipher3
# If it fails, install build dependencies:
sudo apt install build-essential libsqlcipher-dev python3-dev -y
pip3 install pysqlcipher3
```

Create the encrypted database:

```
mkdir -p ~/resqsense && cd ~/resqsense
sqlcipher resqsense.db
-- Inside SQLCipher:
PRAGMA key = 'SET_ENCRYPTION_KEY_HERE';
.read sqlite_schema_rpi.sql
.tables
.quit
```

Access from Python:

```
from pysqlcipher3 import dbapi2 as sqlite
conn = sqlite.connect('resqsense.db')
conn.execute("PRAGMA key = 'KEY_HERE'")
cursor = conn.execute("SELECT * FROM role")
for row in cursor:
    print(row)
conn.close()
```

#### Security Verification

To confirm that encryption works, try opening with regular sqlite3: `sqlite3 resqsense.db ".tables"` should return "Error: file is not a database". Only sqlcipher with the correct key can open the file.

### 10.2 PC (PostgreSQL + pgcrypto)

Install PostgreSQL (Linux):

```
sudo apt update
sudo apt install postgresql postgresql-contrib -y
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

Create database and users:

```
sudo -u postgres psql
CREATE DATABASE resqsense;
CREATE USER resqsense_dev WITH PASSWORD 'CHANGE_DEV_PASSWORD';
CREATE USER resqsense_leader WITH PASSWORD 'CHANGE_LEADER_PASSWORD';
CREATE USER resqsense_sync WITH PASSWORD 'CHANGE_SYNC_PASSWORD';
GRANT ALL PRIVILEGES ON DATABASE resqsense TO resqsense_dev;
GRANT CONNECT ON DATABASE resqsense TO resqsense_leader;
GRANT CONNECT ON DATABASE resqsense TO resqsense_sync;
\q
```

Run the schema and assign permissions:

```
psql -U resqsense_dev -d resqsense -f postgresql_schema_pc.sql
# Assign permissions by role:
```

```
psql -U resqsense_dev -d resqsense
GRANT SELECT, INSERT, UPDATE ON team_leader, operator,
    session, session_operator, alert TO resqsense_leader;
GRANT SELECT ON biometric_data, location_data, role TO resqsense_leader;
GRANT INSERT ON audit_log, failed_login_attempt TO resqsense_leader;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO resqsense_leader;
GRANT INSERT ON biometric_data, location_data, alert,
    audit_log, session, session_operator TO resqsense_sync;
GRANT SELECT ON role, operator, team_leader, session TO resqsense_sync;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO resqsense_sync;
```

#### Configure internal network access:

```
Edit postgresql.conf: listen_addresses = '*'
Edit pg_hba.conf and add:
host resqsense resqsense_dev 192.168.1.0/24 md5
host resqsense resqsense_leader 192.168.1.0/24 md5
host resqsense resqsense_sync 192.168.1.0/24 md5
Restart: sudo systemctl restart postgresql
```

### 10.3 Testing Connectivity

```
# On the Raspberry Pi:
sudo apt install postgresql-client -y
psql -h 192.168.1.XXX -U resqsense_sync -d resqsense -c "SELECT 1;"
```

### 10.4 Audit Log Examples

In the Python application, log actions as follows:

```
def log_action(conn, user_id, role_id, action,
               target_table=None, target_id=None):
    conn.execute(
        """INSERT INTO audit_log
            (user_id, role_id, action, target_table, target_id)
            VALUES (?, ?, ?, ?, ?)""",
        (user_id, role_id, action, target_table, target_id)
    )
    conn.commit()

# Examples:
log_action(conn, 1, 2, 'LOGIN_SUCCESS', 'team_leader', '1')
log_action(conn, 1, 2, 'SESSION_START', 'session', 'session_001')
log_action(conn, 1, 2, 'OPERATOR_ADDED', 'session_operator', 'op_5')
log_action(conn, 1, 2, 'BIOMETRIC_VIEWED', 'biometric_data', 'op_5')
log_action(conn, 1, 2, 'SESSION_END', 'session', 'session_001')
```

#### Final Security Notes

|                          |                                                                    |
|--------------------------|--------------------------------------------------------------------|
| <b>Encryption keys</b>   | Store in environment variables or .env file, NEVER in source code. |
| <b>Passwords</b>         | Change ALL example passwords before using in production.           |
| <b>Internal network</b>  | Adjust the 192.168.1.0/24 range to match your actual network.      |
| <b>PostgreSQL backup</b> | Take regular backups with pg_dump (data remains encrypted).        |
| <b>RPi SSD</b>           | Use an SSD or USB drive instead of SD card for greater durability. |
| <b>Data retention</b>    | Define a data deletion policy (e.g. annual CRON job).              |

